

HW1: Object Detection for Group-housed Swine

1. Model Description

The model is designed for the task of pig detection in images, utilizing the robust and widely adopted Faster R-CNN (Region-based Convolutional Neural Network) framework.

1.1. Core Components

Backbone Feature Extractor: This work utilizes a **ResNet50_v2** network as the backbone. To leverage transfer learning, the weights of this backbone are initialized from a model pre-trained on the ImageNet (IMAGENET1K_V2) dataset. This allows the model to start with a strong, generalized understanding of visual features.

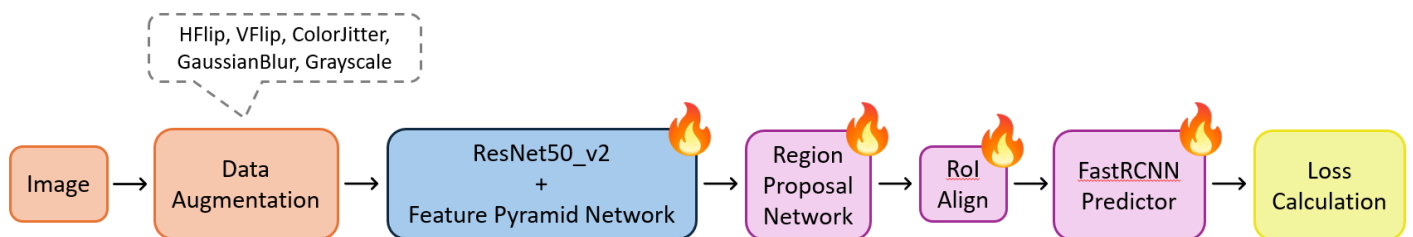
Feature Pyramid Network (FPN): An FPN is integrated with the backbone. It generates multi-scale feature maps, which are crucial for accurately detecting objects (pigs) of various sizes within an image.

Region Proposal Network (RPN): This network scans the feature maps generated by the FPN and proposes a set of candidate bounding boxes, known as "regions of interest" (RoIs), that are likely to contain an object.

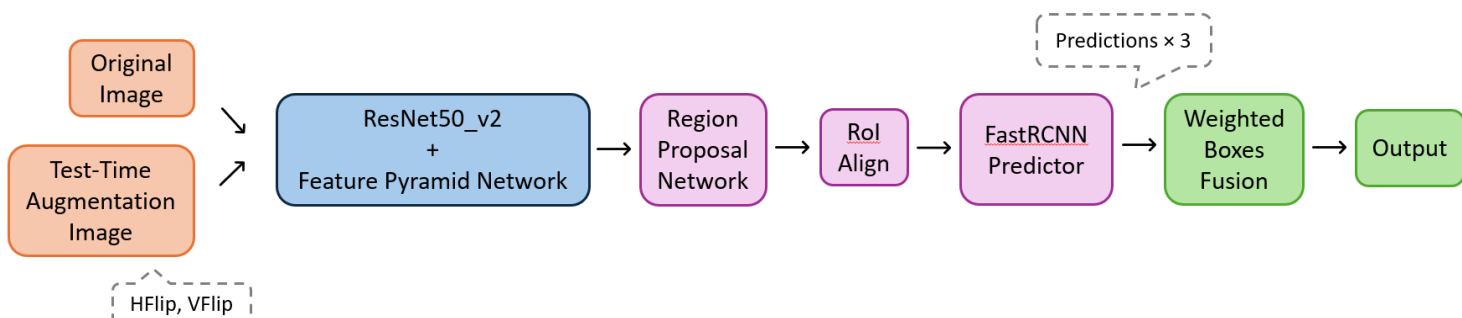
RoI Head (Fast R-CNN Detector): For each proposed RoI, the RoI Align layer extracts a fixed-size feature map. These features are then fed into the final detection head, which performs two tasks. **(1) Classification:** Determines if the RoI contains a pig or is background. **(2) Bounding Box Regression:** Refines the coordinates of the RoI to more accurately fit the detected pig.

1.2. Architecture Illustration

➤ Training Pipeline



➤ Inference Pipeline



1.3. Modifications

The **final classification and regression head was replaced with a new FastRCNNPredictor layer**. This was necessary to match the number of output classes for our task, which is 2 (1 for "pig" and 1 for the "background").

2. Implementation Details

2.1. Dataset and Preprocessing

The dataset was first split into **training and validation sets with a ratio of 80% to 20%**, respectively, using a fixed random seed for reproducibility. All input images were converted from PIL Images to PyTorch Tensors, and pixel values were scaled to the [0, 1] range.

2.2. Data Augmentation

To improve model generalization and prevent overfitting, a series of **custom composite transformation techniques were applied** to the training set.

Random Horizontal Flip: Applied with a probability of 50%.

Random Vertical Flip: Applied with a probability of 30%.

Color Jitter: Randomly changes the brightness, contrast, saturation, and hue of the image.

Gaussian Blur: Applied with a probability of 20% using a 5x5 kernel.

Random Grayscale: Converts the image to grayscale with a probability of 10%.

2.3. Hyperparameters

Epochs	Batch Size	Optimizer	Learning Rate Scheduler	Learning Rate	Weight Decay
70	8	AdamW	StepLR	0.0005	0.0001

2.4. Loss Functions

The Faster R-CNN model inherently combines multiple losses:

Classification Loss (Cross-Entropy) for object presence.

Bounding Box Regression Loss (Smooth L1) for localization accuracy.

RPN Losses for region proposal generation.

The total loss is the weighted sum of all components, computed as:

$$L_{total} = L_{cls} + L_{bbox} + L_{rpn_cls} + L_{rpn_bbox}$$

2.5. Training and Evaluation Strategies

My training process was structured into a two-phase approach, controlled by dev and final modes, to ensure robust model development and optimal final performance.

Phase 1: Development Mode (--mode dev)

The primary purpose of this mode was for experimentation, hyperparameter tuning, and selecting the best model configuration.

Model Selection: The model was trained on the 80% training portion, and its performance was periodically evaluated on the 20% validation set. A model checkpoint “best_model.pth” was saved only when the validation mAP score showed an improvement, ensuring that the best-performing version was always retained.

Early Stopping: An early stopping mechanism was employed to halt the training process if the validation mAP did not improve for 10 consecutive evaluation cycles. This strategy prevents overfitting and saves computational resources.

Phase 2: Final Training Mode (--mode final)

Once the optimal model checkpoint was identified and saved from the development phase, this mode was used to train the final model for submission.

Initialization: The process begins by loading the weights from the “best_model.pth” that was saved during the dev mode.

Full-Dataset Training: The model then continues its training on the entire 100% of the original training data (the previous 80% training and 20% validation sets combined). This strategy leverages all available labeled data to maximize the model's robustness and performance before it is used to generate predictions on the final test set.

Fine-Tuning Approach

Throughout both phases, a full fine-tuning strategy was adopted. The entire model, including the pre-trained ResNet50_v2 backbone, was trained (i.e., no layers were frozen). This allows the feature extractor to adapt its learned features more specifically to the pig dataset.

Evaluation Metric

The primary metric used for model selection in the development phase was the mean Average Precision (mAP). This was calculated on the validation set over a range of Intersection over Union (IoU) thresholds, from 0.5 to 0.95, with a step of 0.05.

3. Results Analysis

3.1. Quantitative Improvements Through Multiple Experiments

Step 1. Each experimental configuration was manually adjusted to identify the combination that yielded the highest publicly available score in the Kaggle competition, **changing only one parameter per experiment**.

Step 2. Record the average loss value per epoch and the mAP on the validation set during each run.

Step 3. Compare the model results across different configuration combinations.

Experiment	Configuration	Public Score
Case 1	backbone: ResNet50, epochs: 50, optimizer: SGD, learning rate scheduler: StepLR, learning rate: 0.0005, weight decay: 0.0001	0.19608
Case 2	backbone: ResNet50, epochs: 50, optimizer: SGD, learning rate scheduler: StepLR, learning rate: 0.0005, weight decay: 0.0001, custom augmentation	0.24378
Case 3	backbone: ResNet50, epochs: 70 , optimizer: SGD, learning rate scheduler: StepLR, learning rate: 0.0005, weight decay: 0.0001, custom augmentation	0.24435
Case 4	backbone: ResNet50, epochs: 70, optimizer: AdamW , learning rate scheduler: StepLR, learning rate: 0.0005, weight decay: 0.0001, custom augmentation	0.28161
Case 5	backbone: ResNet50_v2 , epochs: 70, optimizer: AdamW, learning rate scheduler: StepLR, learning rate: 0.0005, weight decay: 0.0001, custom augmentation	0.29289
Case 6	backbone: ResNet50_v2, epochs: 70, optimizer: AdamW, learning rate scheduler: StepLR, learning rate: 0.0005, weight decay: 0.0001, custom augmentation, TTA + WBF (IoU threshold = 0.60, score threshold = 0.3)	0.30648
Case 7	backbone: ResNet50_v2, epochs: 70, optimizer: AdamW, learning rate scheduler: StepLR, learning rate: 0.0005, weight decay: 0.0001, custom augmentation, TTA + WBF (IoU threshold = 0.55, score threshold = 0.3)	0.30966

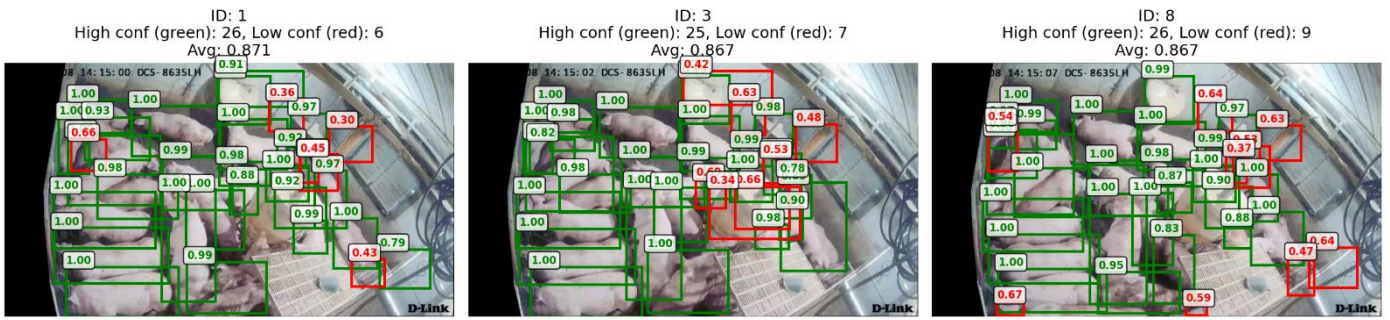
To further improve the final predictions on the test set, two techniques were employed during inference:

Test-Time Augmentation (TTA): Predictions were generated for the original image, its horizontal flip, and its vertical flip.

Weighted Boxes Fusion (WBF): The multiple sets of predictions from TTA were merged using the WBF algorithm, which produces more robust and accurate final bounding boxes than traditional Non-Maximum Suppression.

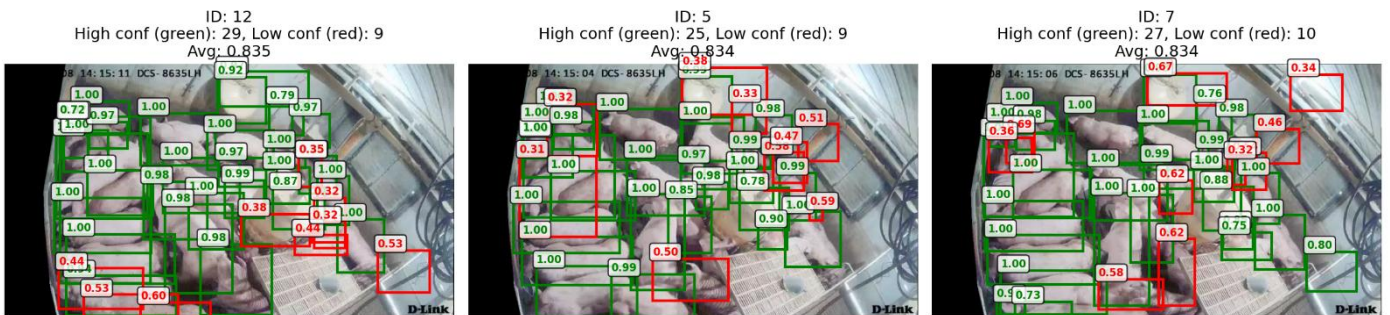
3.2. Visualizations

Success Cases (Better Predictions)



The success cases highlight the model's strong performance and reliability under favorable conditions where pigs are clearly visible with minimal occlusion. In these examples, the model correctly identifies the majority of individual pigs with very high confidence scores, as indicated by the high proportion of green (high-confidence) bounding boxes to red (low-confidence) ones. The average detection confidence across these images is consistently high, approximately 0.87. This demonstrates that the model has effectively learned the key visual features of the target objects and performs robustly when subjects are well-defined.

Failure Cases (Worse Predictions)



The failure cases provide critical insights into the model's limitations, particularly in scenarios involving severe occlusion and object crowding. In these images, where pigs are densely packed, the model's ability to distinguish individuals is compromised. This results in a noticeable increase in the number of low-confidence detections (red boxes), especially in the most overlapped areas. The performance degradation is also reflected in a lower average detection confidence of approximately 0.83. These examples are essential for error analysis, clearly identifying that the model's primary weakness is its handling of highly occluded subjects and suggesting a clear direction for future improvements.

4. Short Conclusion

This project presented a comprehensive pipeline for pig detection using a Faster R-CNN model with a ResNet50_v2-FPN backbone. By leveraging transfer learning, extensive data augmentation, and a full fine-tuning strategy, the model achieved an outstanding public score of 0.30966 on the Kaggle competition. Post-processing techniques like TTA and WBF were implemented to enhance the final inference results.